

디자인-코드 핸드오프 워크플로우

디자이너, 개발자, 에이전트가 Figma 디자인을 코드로 옮기기 위한 협업 흐름이다. 이 워크플로우는 세 스킬을 순서대로 연결한다.

- `design-md`: 디자인 시스템과 token artifact를 만든다.
- `components`: Figma component와 code component의 mapping contract를 관리한다.
- `figma-implement`: 완성된 Figma component 또는 page frame을 UI code로 구현한다.

스킬별 상세 설명:

스킬	설명 문서
<code>design-md</code>	<code>docs/skills/design-md.md</code>
<code>components</code>	<code>docs/skills/components.md</code>
<code>figma-implement</code>	<code>docs/skills/figma-implement.md</code>

전제 조건

- Figma 파일은 가능하면 2개로 분리한다.
 - File A / File 1**: 디자인 시스템, foundations, variables, styles, component library
 - File B / File 2**: 실제 제품 페이지, File A library component instance로 구성
- 디자이너는 File 1 component를 library에 publish한다.
- File 2의 제품 페이지는 detach된 raw frame이 아니라 File 1 component instance를 우선 사용한다.
- 에이전트는 Figma MCP에 접근할 수 있어야 한다.
- 앱은 Tailwind v4 기준으로 `tailwind.theme.css`를 import할 수 있어야 한다.
- 별도 자동 매핑 기능은 사용하지 않는다. Figma component와 code component의 공식 매핑 문서는 `COMPONENTS.md`다.

전체 흐름

Phase 1. design-md

File 1 foundations/design system 분석

-> DESIGN.md, tailwind.theme.css, tokens.studio.global.json

-> handoff mode라면 COMPONENTS.md 초안도 생성

Phase 2. components

File 1 component library 분석

-> COMPONENTS.md mapping table 누적 갱신

Phase 3. 제품 페이지 디자인

디자이너가 File 2에서 File 1 component instance로 page frame 완성

-> node-id가 포함된 frame URL 전달

Phase 4. figma-implement

File A component 또는 File B frame과 handoff artifacts 소비

-> component-level 또는 page-level code 구현

Phase 1 - 디자인 시스템 수립

트리거: 프로젝트 시작 또는 색상, 타이포그래피, spacing, radius 등 foundation token 변경 시 **스킬:**

`design-md` **입력:** File 1의 Foundations 또는 Design System 페이지 URL

개발자 -> 에이전트

File 1 Foundations URL: <url>

design-md 스킬 실행.

DESIGN.md, tailwind.theme.css, tokens.studio.global.json을 생성해줘.

구현까지 이어지는 handoff package가 필요하다면 다음처럼 요청한다.

File 1 Design System URL: <url>

이 디자인 시스템을 개발 핸드오프용으로 준비해줘.

DESIGN.md, tailwind.theme.css, tokens.studio.global.json,

COMPONENTS.md를 생성해줘.

에이전트 산출물

파일	소비자	역할
DESIGN.md	에이전트, 개발자	디자인 시스템 source of truth
tailwind.theme.css	개발자, 앱	Tailwind v4 @theme token CSS
tokens.studio.global.json	디자이너	Tokens Studio global token-set JSON
COMPONENTS.md	에이전트, 개발자	handoff mode에서 생성되는 component contract 초안

검토

- 개발자: tailwind.theme.css 를 앱의 Tailwind entry stylesheet에 import한다.
- 디자이너: tokens.studio.global.json 을 Tokens Studio의 global token set JSON View에 붙여 넣고 token 이름과 역할을 확인한다.
- 개발자: handoff mode에서 생성된 COMPONENTS.md 를 보고 실제 코드 구조와 맞는지 확인한다.

주의점

- DESIGN.md 의 ## Components 에는 component reuse policy만 둔다.
- 실제 component path, mapping, variant note는 COMPONENTS.md 에 둔다.
- Figma component와 code component의 상세 매핑은 COMPONENTS.md 에 둔다.

Phase 2 - 컴포넌트 계약 수립

트리거: Figma component 추가, 수정, variant/property 변경, code path 확정 시 **스킬:** component

입력: File 1의 개별 component URL 또는 Components page URL

Phase 2는 반복 루프다. component가 추가되거나 수정될 때마다 다시 실행한다.

디자이너: Figma component 제작 및 publish

↓

개발자: components 스킴 실행

↓

에이전트: COMPONENTS.md 추가 또는 갱신

↓

개발자 또는 에이전트: figma-implement로 pending code component 구현

↓

COMPONENTS.md의 pending path를 실제 경로로 갱신

Step 2-1. 디자이너: Figma component 제작

- Phase 1에서 import한 token을 사용한다.
- raw color, raw font, raw spacing을 직접 쓰지 않는다.
- variant/property 이름은 code props와 최대한 맞춘다.
 - 예: `variant=primary`, `size=md`, `state=disabled`
- 완성 후 library에 publish한다.
- 제품 페이지에서는 component instance를 detach하지 않는다.

Step 2-2. 개발자 -> 에이전트: components 실행

File 1 component URL 또는 Components page URL: <url>

components 스킴 실행.

COMPONENTS.md를 갱신해줘.

기존 entry는 삭제하지 말고, 코드 경로가 확인되지 않으면 pending으로 뒀.

COMPONENTS.md 권장 구조

`COMPONENTS.md` 는 `components` 스킴이 계속 관리하는 누적 contract다. handoff mode에서 `design-md` 가 초안을 만들 수 있지만, 이후 owner는 `components` 다.

```
# Component Contract
```

```
## Lookup Order
```

1. COMPONENTS.md mapped Code Path
2. components/ui
3. components/sections
4. route-local components
5. new component - reason required

```
## Components
```

```
| Figma Name | Code Path | Variants/Notes |  
|-----|-----|-----|  
| Button | components/ui/Button.tsx | variant=primary/ghost, size=sm/md |
```

```
## Primitive Components
```

```
| Code Path | Role | Notes |  
|-----|-----|-----|  
| components/ui/Button.tsx | button primitive | Used by Figma Button |
```

```
## New Component Rule
```

New shared components are allowed only when existing components cannot express the required UI through props, variants, slots, or composition. Every new component must include the reason existing components are insufficient.

Step 2-3. pending code component 구현

`COMPONENTS.md`에 `pending` entry가 있으면 구현 전 경로를 확정해야 한다.

figma-implement 스킴을 사용해서

`COMPONENTS.md`의 `pending` 항목을 보고,

Figma URL에서 component structure와 variants를 파악해서 구현해줘.

구현 후 `COMPONENTS.md`의 Code Path를 실제 경로로 갱신해줘.

예시:

```
| Button | components/ui/Button.tsx | variant=primary/ghost, size=sm/md |
```

주의점

- Figma component 이름, `COMPONENTS.md` entry 이름, code component 이름은 가능하면 일치시킨다.
- 이름이 다르면 `Variants/Notes`에 mapping note를 남긴다.

- `pending` 은 실제 미구현 또는 미확인 상태에만 사용한다.
- 기존 mapping은 의도 없이 삭제하지 않는다.

Phase 3 - 제품 페이지 디자인

트리거: 구현할 기능 또는 페이지가 확정되었을 때 **담당:** 디자이너 **입력:** File 1 library, Phase 2에서 관리되는 component contract

디자이너: File 2에서 작업

- File 2에서 File 1 library를 활성화한다.
- 페이지는 File 1 component instance 중심으로 구성한다.
- 화면별 state와 responsive behavior를 필요한 만큼 포함한다.
- 실제 구현 가능한 수준으로 spacing, text wrapping, asset crop을 정리한다.
- 완성 후 개발자에게 node-id가 포함된 frame URL을 전달한다.

전달물

Figma URL: <File 2 frame URL with node-id>
Target route: <route-or-file-path>
Notes: <필요한 interaction/state/asset 설명>

Phase 4 - 구현

트리거: File A component 구현 대상 또는 File B 제품 페이지/frame이 완성되고 target path가 정해졌을 때 **스킬:** `figma-implement` **입력:** File A component URL 또는 File B frame URL, target path, `DESIGN.md`, `tailwind.theme.css`, `COMPONENTS.md`

실행 전 체크

- `DESIGN.md`가 있다.
- `COMPONENTS.md`가 있다.
- `tailwind.theme.css`가 앱의 global Tailwind stylesheet에 import되어 있다.
- Figma URL에 `node-id`가 있다.
- target component, route, 또는 file path가 있다.

개발자 -> 에이전트

```
Figma URL: <File A component URL 또는 File B frame URL with node-id>

Target: components/sections/PricingCard.tsx 또는 app/[route]/page.tsx

Use DESIGN.md, tailwind.theme.css, COMPONENTS.md.

Do not create new shared components unless existing ones cannot express the design.
```

에이전트 실행

- 1. Figma MCP로 component/frame structure와 assets를 수집한다.
- 2. COMPONENTS.md Lookup Order를 따른다.
- 3. 기존 component를 props, variants, slots, composition으로 표현할 수 있는지 확인한다.
- 4. component-level 구현이면 matching entry를 pending 에서 실제 path로 갱신한다.
- 5. page-level 구현이면 COMPONENTS.md 에 등록된 component를 먼저 재사용한다.
- 6. 기존 component로 불가능할 때만 새 component 이유를 기록하고 생성한다.
- 7. target component, route, 또는 file을 구현한다.
- 8. 구현된 path, COMPONENTS.md 갱신 내역, blocker를 보고한다.

완료 기준

- 구현된 style value가 DESIGN.md 또는 tailwind.theme.css token으로 추적 가능하다.
- shared component는 COMPONENTS.md 에 있거나 생성 이유가 기록되어 있다.

스킬 요약

스킬	입력	출력	실행 빈도	다음 단계
design-metadata	File 1 Foundations/Design System URL	DESIGN.md, tailwind.theme.css, tokens.studio.global.json, handoff mode의 COMPONENTS.md	프로젝트 시작 및 token 변경 시	components
component-templates	File 1 component URL 또는 Components page URL	COMPONENTS.md mapping contract	component 변경 시마다	File 2 제품 페이지 디자인 또는 figma-importment
figma-importment	File 1 component URL 또는 File 2 frame URL with node-id, target path, handoff artifacts	component-level 또는 page-level code	컴포넌트/페이지 구현 시마다	구현 검토 또는 Phase 2 loop

이름 일관성 규칙

`COMPONENTS.md`로 에이전트가 component를 안정적으로 matching하려면 이름이 일치해야 한다.

```
Figma component: Button
COMPONENTS.md:    Button -> components/ui/Button.tsx
Code file:        Button.tsx
```

세 곳의 이름이 다르면 에이전트가 추측에 의존한다. 이름 불일치가 발견되면 `components` 스킴이 `COMPONENTS.md`의 `Variants/Notes`에 mapping note를 남기고, 디자이너 또는 개발자에게 이름 정렬을 요청한다.

루프백 규칙

구현 중 다음 문제가 발견되면 이전 phase로 돌아간다.

발견된 문제	돌아갈 phase	이유
token이 부족하거나 Figma token과 code token이 다름	Phase 1	<code>DESIGN.md</code> 와 token artifacts 갱신 필요
Figma component mapping이 없거나 <code>pending</code> 임	Phase 2	<code>COMPONENTS.md</code> 갱신 또는 code component 구현 필요
기존 component로 표현할 수 없는 새 shared component 필요	Phase 2	새 component 등록 및 재사용 contract 갱신 필요
제품 page frame이 구현 불가능한 raw layout임	Phase 3	library instance 기반으로 design 정리 필요

운영 체크리스트

- `design-md` 실행 후 `@google/design.md lint`가 통과했는가?
- `tailwind.theme.css`가 fresh export와 일치하는가?
- `tokens.studio.global.json`이 wrapper 없는 token-set JSON인가?
- `tailwind.theme.css`가 앱 stylesheet에 import되었는가?
- `COMPONENTS.md`에 필요한 Figma component가 모두 있는가?
- `pending` code path가 구현 전에 정리되었는가?
- File 2 page frame은 File 1 library component instance를 사용했는가?
- `figma-implement` 후 구현된 path와 `COMPONENTS.md` 갱신 내역이 보고되었는가?